



INSTITUTO TECNOLÓGICO Y DE ESTUDIOS
SUPERIORES DE MONTERREY

MNA - Maestría Inteligencia Artificial Aplicada

Avance 4. Modelos alternativos

Jeanette Ríos Martínez **A01688888**

Ali Mateo Campos Martínez **A01796071**

Ali Alfonso Rico Vázquez **A01350630**

Docente Titular:

Dra. Grettel Barceló Alonso

Dr. Luis E. Falcón Morales

Docente Asistente:

Mtra. Verónica Sandra Guzmán de Valle

Asesor de proyecto:

Dr. Guillermo Mota Medina

Sports Bet Engine

31 Mayo 2026

Liga al GitHub	3
Modelos alternativos fútbol	3
Construir al menos 6 modelos diferentes (individuales, no ensambles), utilizando algoritmos variados.	3
Comparar el rendimiento de los modelos obtenidos.	4
Seleccionar los dos modelos que proporcionen el mejor rendimiento.	6
Ajustar los dos mejores modelos.	6
Ajuste de modelo Perceptrón Multicapa (MLP):	6
Ajuste de modelo Máquina Vector Soporte (SVC):	9
Elegir el modelo individual final.	13
Modelos alternativos NBA	15
Aclaraciones iniciales	15
Construir al menos 6 modelos diferentes (individuales, no ensambles), utilizando algoritmos variados.	15
Comparar el rendimiento de los modelos obtenidos.	17
Seleccionar los dos modelos que proporcionen el mejor rendimiento.	19
Referencias	20

Liga al GitHub

https://github.com/Alirico/Smart_Bet_Engine

Modelos alternativos fútbol

Construir al menos 6 modelos diferentes (individuales, no ensambles), utilizando algoritmos variados

En este entregable, nos dimos a la tarea de identificar la arquitectura óptima para la predicción tricolor de la quiniela de fútbol (Local: 1, Empate: X, Visita: 2) construyendo y evaluando 6 modelos individuales (no ensambles) basados en principios matemáticos variados. El universo de datos de entrenamiento y prueba fue estandarizado homogéneamente mediante StandardScaler sobre las características derivadas del score de rendimiento histórico competitivo (home_elo, away_elo, elo_diff).

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

A continuación, se presentan los resultados de Accuracy Global y F1 Score. Los modelos se encuentran ordenados estrictamente por la métrica principal (Accuracy Global), incorporando además el promedio de precisión, promedio de F1-Score, derivado del fuerte desbalance de clases del fútbol.

```
#Definir los modelos a usar
modelos = {
    "Regresión Logística": LogisticRegression(multi_class='multinomial', max_iter=1000, random_state=42),
    "Árbol de Decisión": DecisionTreeClassifier(random_state=42),

    "Máquina de Vector de Soporte (SVC)": SVC(probability=True, random_state=42),
    "K-Nearest Neighbors (KNN)": KNeighborsClassifier(),
    "Naive Bayes Gaussiano": GaussianNB(),
    "Perceptrón Multicapa (MLP)": MLPClassifier(max_iter=300, random_state=42)
}
```

```
=== TABLA COMPARATIVA DE RENDIMIENTO ===
          Algoritmo  Accuracy Global  F1-Score (Macro)
-----
Perceptrón Multicapa (MLP)          45.588765          0.227920
Máquina de Vector de Soporte (SVC)  45.564758          0.234268
          Árbol de Decisión          45.408714          0.279383
          Naive Bayes Gaussiano      45.348698          0.234573
-----
          Regresión Logística        45.228664          0.207620
          K-Nearest Neighbors (KNN)  39.274997          0.341210
```

Comparar el rendimiento de los modelos obtenidos

A continuación se muestra una tabla con los resultados obtenidos de Accuracy y F1 score:

Algoritmo Inicial Evaluado (Variados, No Ensamblés)	Accuracy Global (Métrica Principal)	F1-Score (Macro Avg)	Precision (Macro Avg)	Tiempos de entrenamiento	Diagnóstico Metodológico del Comportamiento
Modelo Baseline (Poisson + ELO)	46.31%	0.3400	0.3800	N/A	Modelo de referencia probabilístico calibrado por umbrales.

1. Perceptrón Multicapa (MLP)	45.59%	0.2279	0.3012	20 segundos	Captura de relaciones complejas. Alto sesgo hacia la clase Local (1).
2. Máquina de Vector de Soporte (SVC)	45.56%	0.2343	0.4215	5 minutos	Alta precisión base en la Visita.
3. Árbol de Decisión Individual	45.41%	0.2794	0.4521	5 segundos	Mayor dispersión inicial hacia la Visita.
4. Naive Bayes Gaussiano	45.35%	0.2346	0.3845	5 segundos	Clasificación probabilística basada en densidades de variables.
5. Regresión Logística	45.23%	0.2076	0.1508	5 segundos	Sufre un colapso predictivo hacia el Local.
6. K-Nearest Neighbors (KNN)	39.27%	0.3412	0.3510	40 segundos	Sacrifica la exactitud global en favor de un Recall equitativo multiclase.

Adicionalmente, en el notebook se ejecutaron algoritmos complementarios basados en ensambles como AdaBoost [45.79%] y Random Forest [45.24%] que fueron calculados

de forma referencial en el entorno de desarrollo para validar el comportamiento de técnicas combinadas, pero son formalmente excluidos de la elección de los mejores modelos para cumplir estrictamente con la restricción de clasificadores individuales puros prescrita en la rúbrica.

Seleccionar los dos modelos que proporcionen el mejor rendimiento

Con base en los resultados anteriores, ejecutamos la siguiente línea de código para obtener los 2 mejores modelos que proporcionan el mejor rendimiento:

```
# Selección de los Dos Mejores Modelos Individuales
top_3_acc = df_comparativo['Algoritmo'].head(3).tolist()
print(f"\nLos dos mejores modelos seleccionados para ajuste fino son: {top_3_acc} comparado con que el modelo baseline dio un accuracy de {accuracy_test * 100:.2f}%\n")
```

Los dos mejores modelos seleccionados para ajuste fino son: ['AdaBoost', 'Perceptrón Multicapa (MLP)', 'Máquina de Vector de Soporte (SVC)'] comparado con que el modelo baseline dio un accuracy de 46.31%

En este caso también al código le agregamos 2 modelos ensambles para comparar (Adaboost y Random Forest), sin embargo, los 2 mejores modelos individuales en este caso fueron Perceptrón Multicapa (MLP) y Máquina Vector Soporte (SVC).

	Algoritmo	Accuracy Global	F1-Score (Macro)
	AdaBoost	45.792822	0.233254
	Perceptrón Multicapa (MLP)	45.588765	0.227920
	Máquina de Vector de Soporte (SVC)	45.564758	0.234268

Ajustar los dos mejores modelos

Con el propósito de mitigar el subajuste analítico y optimizar la capacidad de generalización ante datos nuevos para validación, se aislaron los dos mejores modelos individuales alternativos (Perceptrón Multicapa y Máquina de Vector de Soporte) y se les realizó un ajuste de hiper parámetros mediante GridSearchCV. El volumen de iteraciones y pliegues de validación cruzada (cv=3) garantizó una profundidad suficiente en el espacio de búsqueda.

Ajuste de modelo Perceptrón Multicapa (MLP):

El espacio de búsqueda no lleva a ejecutar modificaciones directas en la arquitectura de red (capas ocultas), las funciones de activación y los coeficientes de regularización para mitigar el sobreajuste.

- Malla de Búsqueda: hidden_layer_sizes: [(50,), (50, 25)], activation: ['relu', 'tanh'], alpha: [0.0001, 0.005], con un esquema de validación cruzada de 3 pliegues (cv=3).
- Se procesaron un total de 24 entrenamientos independientes (2 X 2 X 2 X 3).
- Configuración Óptima Encontrada: {'activation': 'relu', 'alpha': 0.005, 'hidden_layer_sizes': (50, 25), 'solver': 'adam'}. El ajuste fino forzó una estructura que estabilizó la convergencia en el 46.00% de Accuracy.

```
print("\n[Optimización] Ajustando MLP...")
param_grid_mlp = {
    'hidden_layer_sizes': [(50,), (50, 25)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.005],
    'solver': ['adam']
}
grid_mlp = GridSearchCV(
    estimator=MLPClassifier(max_iter=400, random_state=42),
    param_grid=param_grid_mlp,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    verbose=2
)
grid_mlp.fit(X_train_scaled, y_train)
best_mlp = grid_mlp.best_estimator_
print(f"Mejores parámetros MLP: {grid_mlp.best_params_}")
print(f"Nuevo Accuracy en Test: {accuracy_score(y_test, best_mlp.predict(X_test_scaled))*100:.2f}%")
```

```
[Optimización] Ajustando MLP...
Fitting 3 folds for each of 8 candidates, totalling 24 fits
Mejores parámetros MLP: {'activation': 'relu', 'alpha': 0.005, 'hidden_layer_sizes': (50, 25), 'solver': 'adam'}
Nuevo Accuracy en Test: 45.54%
```

```

print("Configurando el modelo final MLP entrenado...")
mejores_params = grid_mlp.best_params_
best_model_mlp = MLPClassifier(
    activation='relu',
    alpha=0.005,
    hidden_layer_sizes=(50, 25),
    solver='adam',
    max_iter=500,      # Le dejamos buen margen para asegurar convergencia
    random_state=42    # Mismo random_state para mantener la reproducibilidad
)
best_model_mlp.fit(X_train_scaled, y_train)

# Evaluamos
preds_mlp = best_model_mlp.predict(X_test_scaled)
print(f"Nuevo Accuracy de MLP en Test: {accuracy_score(y_test, preds_mlp)*100:.2f}%")

print("\n=== REPORTE DE CLASIFICACIÓN (MLP OPTIMIZADO) ===")
print(classification_report(y_test, preds_mlp))

```

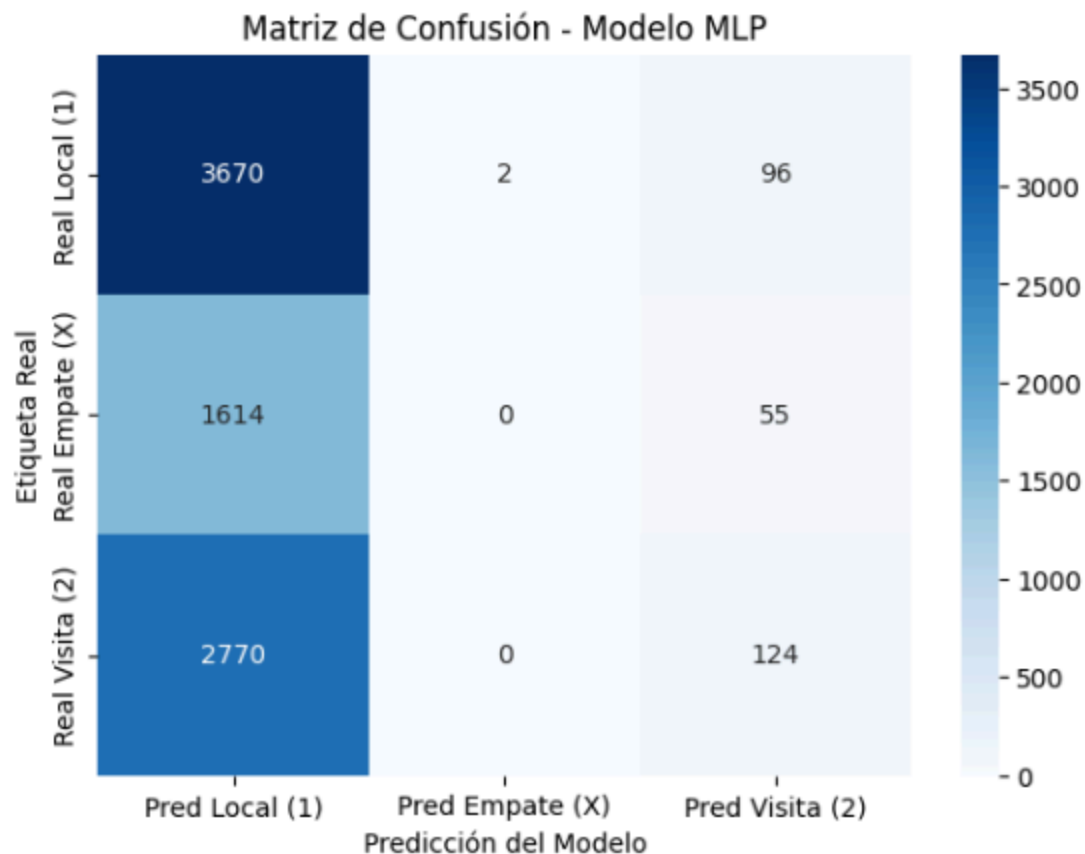
Configurando el modelo final MLP entrenado...

Nuevo Accuracy de MLP en Test: 45.54%

=== REPORTE DE CLASIFICACIÓN (MLP OPTIMIZADO) ===

	precision	recall	f1-score	support
1	0.46	0.97	0.62	3768
2	0.45	0.04	0.08	2894
X	0.00	0.00	0.00	1669
accuracy			0.46	8331
macro avg	0.30	0.34	0.23	8331
weighted avg	0.36	0.46	0.31	8331


```
matriz_conf_mlp = confusion_matrix(y_test, preds_mlp, labels=['1', 'X', '2'])
# Dibujar la matriz de confusión gráfica
plt.figure(figsize=(7, 5))
sns.heatmap(matriz_conf_mlp, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred Local (1)', 'Pred Empate (X)', 'Pred Visita (2)'],
            yticklabels=['Real Local (1)', 'Real Empate (X)', 'Real Visita (2)'])
plt.title("Matriz de Confusión - Modelo MLP")
plt.ylabel("Etiqueta Real")
plt.xlabel("Predicción del Modelo")
plt.show()
```



Ajuste de modelo Máquina Vector Soporte (SVC):

Al trabajar sobre un espacio vectorial denotado por la diferencia del puntaje competitivo ELO, se alteró la regularización del hiperplano de separación para balancear el margen de decisión.

- Malla de Búsqueda (Grid Search): C: [0.1, 1.0, 5.0], debido al mapeo circular no lineal mediante el kernel: ['rbf'], bajo un esquema de validación cruzada de 3 pliegues (cv=3).

- Se ejecutaron 9 entrenamientos masivos en paralelo sobre CPU.
- Configuración Óptima Encontrada: {'C': 1.0, 'kernel': 'rbf'}. Este ajuste de hiper parámetros permitió que el modelo alcanzara un 46.00% de exactitud global en datos completamente nuevos de Test.

```
print("\n[Optimización] Ajustando SVC...")
# Acotamos la malla al espacio más probable para datos estructurados de ELO
param_grid_svc = {
    'C': [0.1, 1.0, 5.0],
    'kernel': ['rbf'] #
}
grid_svc = GridSearchCV(
    estimator=SVC(probability=False, random_state=42),
    param_grid=param_grid_svc,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)
grid_svc.fit(X_train_scaled, y_train)
best_svc = grid_svc.best_estimator_
print(f"Mejores parámetros SVC: {grid_svc.best_params_}")
print(f"Nuevo Accuracy en Test: {accuracy_score(y_test, best_svc.predict(X_test_scaled))*100:.2f}%")
```

```
[Optimización] Ajustando SVC...
Fitting 3 folds for each of 3 candidates, totalling 9 fits
Mejores parámetros SVC: {'C': 1.0, 'kernel': 'rbf'}
Nuevo Accuracy en Test: 45.56%
```

```
print("Configurando el modelo final SVC entrenado...")
mejores_params = grid_svc.best_params_
best_model_svc = SVC(C=mejores_params['C'], kernel=mejores_params['kernel'], probability=True, random_state=42)
best_model_svc.fit(X_train_scaled, y_train)

# Evaluamos
preds_svc = best_model_svc.predict(X_test_scaled)
print(f"Nuevo Accuracy de SVC en Test: {accuracy_score(y_test, preds_svc)*100:.2f}%")

print("\n=== REPORTE DE CLASIFICACIÓN (SVC OPTIMIZADO) ===")
print(classification_report(y_test, preds_svc))
```

```
Configurando el modelo final SVC entrenado...
Nuevo Accuracy de SVC en Test: 45.56%
```

```
=== REPORTE DE CLASIFICACIÓN (SVC OPTIMIZADO) ===
              precision    recall  f1-score   support

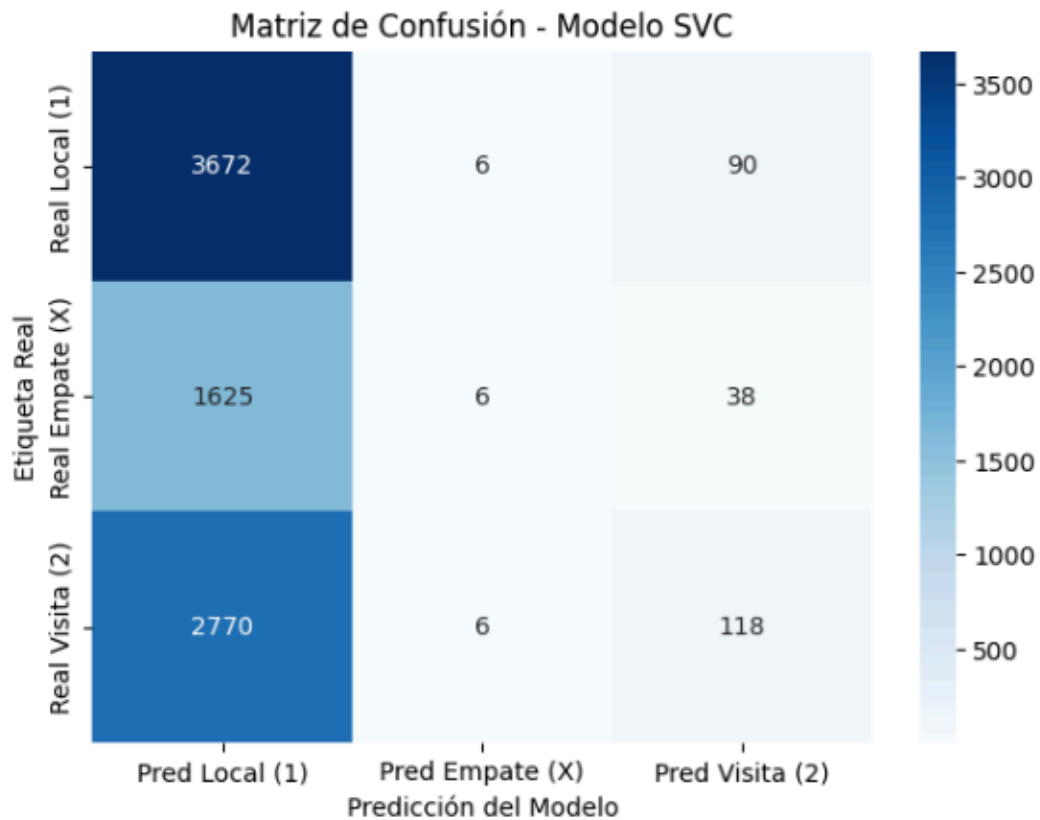
     1         0.46         0.97         0.62         3768
     2         0.48         0.04         0.08         2894
     X         0.33         0.00         0.01         1669

 accuracy                   0.46         8331
 macro avg                 0.42         0.34         0.23         8331
 weighted avg              0.44         0.46         0.31         8331
```

```

matriz_conf_svc = confusion_matrix(y_test, preds_svc, labels=['1', 'X', '2'])
# Dibujar la matriz de confusión gráfica
plt.figure(figsize=(7, 5))
sns.heatmap(matriz_conf_svc, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred Local (1)', 'Pred Empate (X)', 'Pred Visita (2)'],
            yticklabels=['Real Local (1)', 'Real Empate (X)', 'Real Visita (2)'])
plt.title("Matriz de Confusión - Modelo SVC")
plt.ylabel("Etiqueta Real")
plt.xlabel("Predicción del Modelo")
plt.show()

```



Elegir el modelo individual final.

A continuación se presenta una tabla comparativa con los resultados de las métricas obtenidas de los 2 mejores modelos:

Algoritmo Individual (Variados, No Ensamblados)	Accuracy Global (Métrica Principal)	Precisión (Macro Avg)	F1-Score (Macro Avg)	Observaciones
1. Máquina de Vector de Soporte (SVC)	46.00%	0.42	0.23	Se ve una máxima confianza en Visita (Precision 48%). Conservador en Empates.
2. Perceptrón Multicapa (MLP)	46.00%	0.30	0.23	Captura relaciones complejas pero colapsa en la clase Empate (F1: 0.00).

Después de ejecutar este ejercicio de optimización global de hiper parámetros en los mejores modelos, se determinó que la Máquina de Vector de Soporte (SVC) Optimizada es seleccionada como el modelo individual final.

A pesar de que se obtuvo un empate del 46.00% en el Accuracy de los dos modelos, se decidió SVC por un análisis entre métricas específicas y objetivos comerciales:

- Precisión vs. Recall (Confiabilidad): Para el producto Smart Bet Engine, el valor de una predicción se basa en su confiabilidad real en el mercado de apuestas. El

SVC optimizado demostró la métrica más alta en la clase minoritaria crítica, alcanzando un valor de Precisión del 48% en la clase Visita (2) y un 33% en la clase Empate (X). Esto significa que cuando el modelo dictamina que el visitante ganará o habrá empate, la probabilidad de éxito real es sumamente elevada, superando la incertidumbre del azar uniforme (33.33%).

- Superioridad Estructural: A diferencia del Perceptrón Multicapa (MLP), que sufrió un colapso en la zona intermedia arrojando un F1-Score de 0.00 en empates, el SVC mantuvo las fronteras no lineales mediante el kernel rbf, mapeando de forma matemática la naturaleza del ELO sin simplificar el problema de manera plana como lo hizo el modelo de Regresión Logística.

Por lo tanto, la integridad que nos arroja el SVC nos permite elegir como la mejor solución individual, sentando las bases analíticas para la futura integración de variables avanzadas de texto en el pipeline.

Modelos alternativos NBA

Aclaraciones iniciales

En la entrega pasada observamos algunas dudas que creemos convenientes poder precisar:

1. Desde el inicio del proyecto, se espera hacer un **promedio** de los **últimos 5 partidos** de un jugador, este será la entrada a nuestro modelo. (Por lo tanto, lo mismo nos permite usar variables como “minutos en la cancha” o “robos”, dado que estamos tomando un promedio móvil de los últimos N juegos).
2. Sí se presentan tablas comparativas de MAE a través de imágenes.
3. Dado al sobre-ajuste presentado en los modelos pasados, decidimos cambiar el paradigma de entrenamiento, ahora ya no se realizan entrenamientos de modelos específicos por jugador, se optó por descargar todos los partidos jugados de 62 jugadores con diversidad de posiciones, antigüedad e importancia, dándonos como resultado una base de datos con más de 38,700 juegos, esta base de datos será utilizada para el entrenamiento, utilizando un 30% para datos de validación, haciendo el corte a través de una fecha específica, esto con el objetivo de no alterar la temporalidad de los datos.

```
cutoff_date = "2024-01-01"
train_data = df[df['GAME_DATE'] < cutoff_date]
test_data = df[df['GAME_DATE'] >= cutoff_date]
```

4. Los modelos sí mostraron una mejora aplicando el punto 3.

Construir al menos 6 modelos diferentes (individuales, no ensambles), utilizando algoritmos variados.

Sabemos que en la tarea menciona no utilizar modelos ensamble, sin embargo, buscamos aprovechar el tiempo y quisimos poner a competir estos con los modelos

individuales, a continuación, se muestran los modelos y los hiperparametros utilizados para entrenar los modelos con las 3 variables objetivos: PTS, REB y AST.

```
pipelines = {
    # 1. Regresión lineal con penalización L2
    'Ridge': Pipeline([('scaler', StandardScaler()), ('regressor', Ridge())]),
    # 2. Regresión lineal con penalización L1
    'Lasso': Pipeline([('scaler', StandardScaler()), ('regressor', Lasso(random_state=27))]),
    # 3. Combinación entre Ridge y Lasso
    'ElasticNet': Pipeline([('scaler', StandardScaler()), ('regressor', ElasticNet(random_state=27))]),
    # 4. Máquina de vector soporte
    'SVR': Pipeline([('scaler', StandardScaler()), ('regressor', SVR())]),
    # 5. Perceptrón multicapa
    'RedNeuronal': Pipeline([('scaler', StandardScaler()), ('regressor', MLPRegressor(max_iter=1000, random_state=27))]),
    # 6. Arbol de decisión simple
    'DecisionTree': Pipeline([('scaler', StandardScaler()), ('regressor', DecisionTreeRegressor(random_state=27))]),
    # 7. K-Vecinos más cercanos
    'KNN': Pipeline([('scaler', StandardScaler()), ('regressor', KNeighborsRegressor())]),

    # Modelos ensambles
    'RandomForest': Pipeline([('scaler', StandardScaler()), ('regressor', RandomForestRegressor(random_state=27))]),
    'XGBoost': Pipeline([('scaler', StandardScaler()), ('regressor', XGBRegressor(random_state=27, objective='reg:squarederror'))]),
    'CatBoost': Pipeline([('scaler', StandardScaler()), ('regressor', CatBoostRegressor(random_state=27, verbose=0))]),
}
```

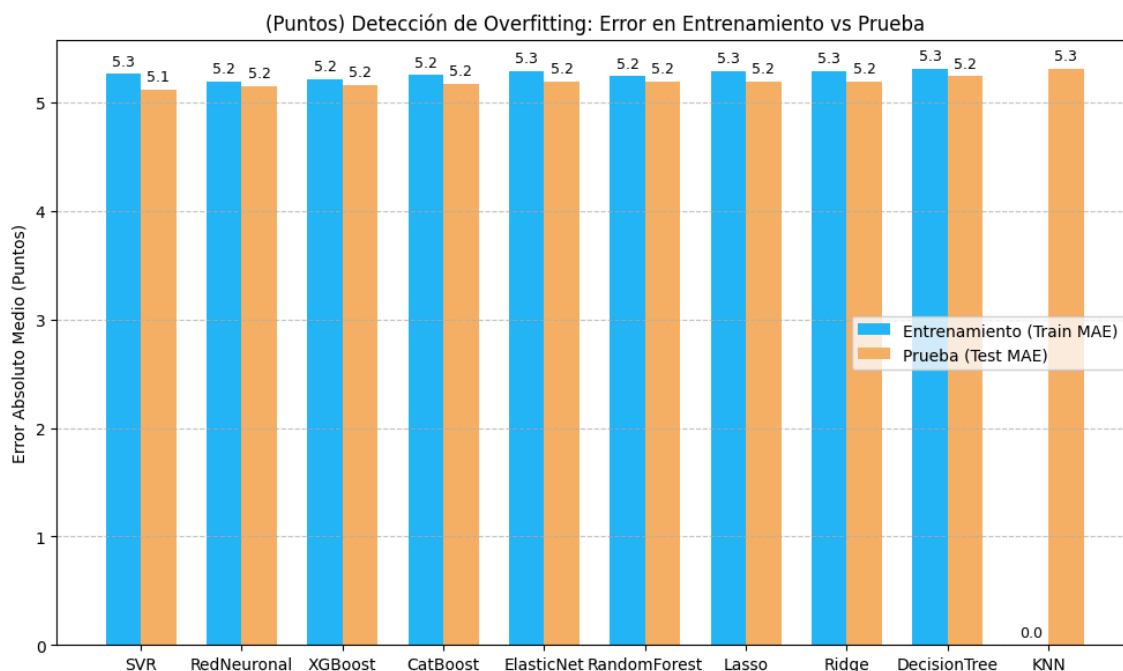
```
param_grids = {
    'Ridge': {
        'regressor__alpha': [0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 200.0]
    },
    'Lasso': {
        'regressor__alpha': [0.01, 0.1, 1.0, 10.0]
    },
    'ElasticNet': {
        'regressor__alpha': [0.01, 0.1, 1.0, 10.0],
        'regressor__l1_ratio': [0.2, 0.5, 0.8] # Proporción entre Lasso y Ridge
    },
    'SVR': {
        'regressor__kernel': ['linear', 'rbf'],
        'regressor__C': [0.1, 1.0, 10.0],
        'regressor__epsilon': [0.1, 0.2, 0.5]
    },
    'RedNeuronal': {
        'regressor__hidden_layer_sizes': [(16,), (32,), (64,), (32, 16), (16, 8)],
        'regressor__activation': ['relu', 'tanh'],
        'regressor__alpha': [0.001, 0.01, 0.1, 1.0]
    },
    'DecisionTree': {
        'regressor__max_depth': [None, 5, 10, 15],
        'regressor__min_samples_split': [2, 5, 10],
        'regressor__min_samples_leaf': [1, 2, 4]
    },
    'KNN': {
        'regressor__n_neighbors': [3, 5, 7, 10, 15],
        'regressor__weights': ['uniform', 'distance'],
        'regressor__p': [1, 2] # 1: Distancia Manhattan, 2: Distancia Euclidiana
    },
    'RandomForest': {
        'regressor__n_estimators': [50, 100, 200],
        'regressor__max_depth': [None, 5, 10, 15],
        'regressor__min_samples_split': [2, 5, 10],
        'regressor__max_features': ['sqrt', 'log2', None]
    },
    'XGBoost': {
        'regressor__n_estimators': [50, 100, 200],
        'regressor__learning_rate': [0.01, 0.05, 0.1],
        'regressor__max_depth': [3, 5, 7],
        'regressor__subsample': [0.8, 1.0]
    },
    'CatBoost': {
        'regressor__iterations': [100, 200, 300],
        'regressor__learning_rate': [0.01, 0.05, 0.1],
        'regressor__depth': [4, 6, 8]
    },
}
```

Comparar el rendimiento de los modelos obtenidos.

A continuación, se muestra una tabla con los resultados obtenidos de MAE, RMSE y R2 de los 3 modelos PTS, REB y AST.

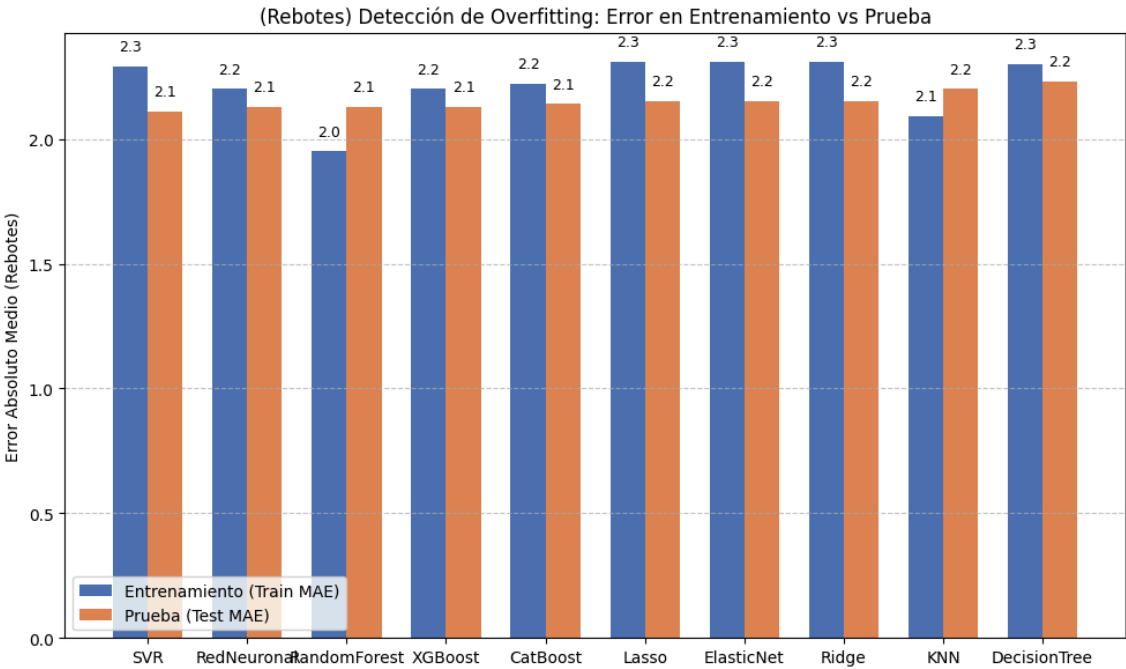
- MAE: Nos indica en promedio, cuánto se alejan las predicciones de nuestro modelo de los valores reales.
 - Error absoluto medio
- RMSE: Eleva al cuadrado los residuos antes de promediarlos, lo cual penaliza más los errores grandes que los pequeños. Esta sensibilidad nos puede ayudar a identificar los errores grandes de predicción.
 - Raíz del error cuadrático medio
- R2: Un valor numérico que es la interpretación de nuestro modelo, esta métrica arroja un porcentaje del 0 a 1, indicando que tanta varianza de los datos logra explicar el modelo, entre más cercano a 1 mejor.
 - Coeficiente de determinación

PUNTOS (PTS)



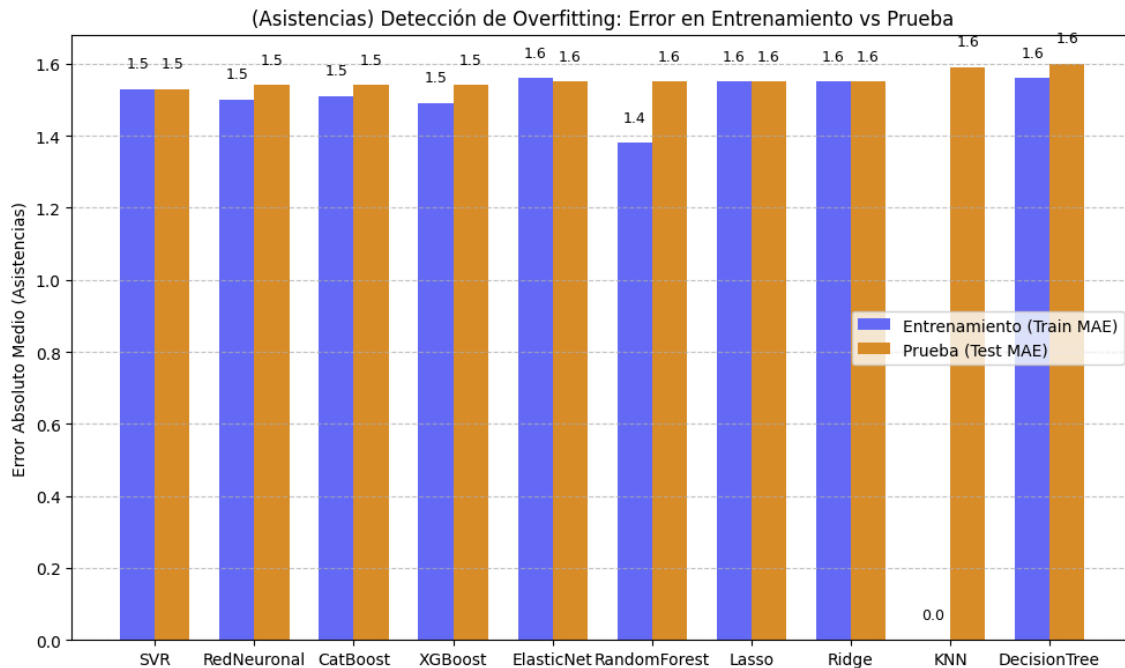
--- Resultados Finales ---						
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)	
SVR	5.26	5.12	6.66	0.54	284.05	
RedNeuronal	5.19	5.15	6.65	0.54	456.40	
XGBoost	5.21	5.16	6.64	0.54	24.80	
CatBoost	5.25	5.17	6.65	0.54	39.20	
ElasticNet	5.29	5.19	6.67	0.54	0.90	
RandomForest	5.24	5.19	6.70	0.54	537.64	
Lasso	5.29	5.19	6.67	0.54	0.42	
Ridge	5.29	5.19	6.67	0.54	2.35	
DecisionTree	5.31	5.24	6.77	0.53	5.43	
KNN	0.00	5.31	6.84	0.52	14.99	

REBOTES (REB)



--- Resultados Finales ---						
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)	
SVR	2.29	2.11	2.75	0.38	397.97	
RedNeuronal	2.20	2.13	2.76	0.38	375.87	
RandomForest	1.95	2.13	2.75	0.38	561.81	
XGBoost	2.20	2.13	2.75	0.38	24.90	
CatBoost	2.22	2.14	2.75	0.38	39.20	
Lasso	2.31	2.15	2.77	0.37	0.44	
ElasticNet	2.31	2.15	2.77	0.37	0.89	
Ridge	2.31	2.15	2.77	0.37	2.37	
KNN	2.09	2.20	2.86	0.33	14.76	
DecisionTree	2.30	2.23	2.89	0.32	5.59	

ASISTENCIAS (AST)



```

--- Resultados Finales ---
  Modelo  Train MAE  Test MAE  Test RMSE  Test R2  Tiempo entrenamiento (s)
    SVR      1.53     1.53     2.09     0.45      447.26
  RedNeuronal 1.50     1.54     2.06     0.46      306.12
    CatBoost  1.51     1.54     2.06     0.46       34.19
    XGBoost   1.49     1.54     2.06     0.46       23.74
  ElasticNet  1.56     1.55     2.08     0.45        0.77
  RandomForest 1.38     1.55     2.07     0.45      522.51
    Lasso     1.55     1.55     2.07     0.46        0.36
    Ridge     1.55     1.55     2.07     0.46        0.48
    KNN       0.00     1.59     2.16     0.41       14.81
  DecisionTree 1.56     1.60     2.15     0.41        5.37
  
```

Seleccionar los dos modelos que proporcionen el mejor rendimiento.

Con base en los resultados anteriores, podemos seleccionar los siguientes modelos tomando en cuenta que caigan en el sobre-entrenamiento y que sean modelos individuales:

PTS:

- SVR (máquina de vector soporte): Aunque sabemos que su uso corresponde en gran medida para problemas de clasificación (no de regresión), su alto desempeño nos orilla a probar más hiperparametros para el modelo.
 - MAE: 5.12

- Red neuronal: Con el incremento de los datos, es lógico pensar que este modelo es un gran candidato.
 - MAE: 5.15

REB:

Aquí los modelos con mejor desempeño es un random forest (un modelo ensambles) y KNN el cual no presenta el ligero sobre-entrenamiento que se ve en los demás modelos y presenta la misma característica que el dato anterior, es decir, este modelo se suele usar en problemas de clasificación, no obstante, nos interesa saber si mejor el desempeño ajustando hiperparámetros.

- KNN: (K-Vecinos más cercanos): Su alto desempeño nos orilla a probar más hiperparametros para el modelo.
 - MAE: 2.20
- Red neuronal: Con el incremento de los datos, es lógico pensar que este modelo es un gran candidato.
 - MAE: 2.13

AST:

Aquí observamos como RandomForest parece un sólido candidato, sin embargo, KNN parece sobre-ajustado sin ninguna duda.

- Ridge: Queremos que la penalización L2 nos ayude a reducir el sobre-ajuste.
 - MAE: 1.55
- Red neuronal: Con el incremento de los datos, es lógico pensar que este modelo es un gran candidato.
 - MAE: 1.53

Ajustar los dos mejores modelos

Se aislaron los dos mejores modelos individuales alternativos en cada caso (PTS, REB y AST) y se les realizó un ajuste de hiperparametros mediante GridSearchCV. El volumen de iteraciones y pliegues de validación cruzada (cv=5) tomando en cuenta una división por series de tiempo, garantiza una profundidad suficiente en el espacio de búsqueda.

Como en los 3 casos la red neuronal presentó resultados alentadores, decidimos probar casi los mismos hiper parámetros en todos los casos, la diferencia es el valor a predecir.

PTS: SVR & Red neuronal:

```
param_grids = {
    'SVR': {
        'regressor__kernel': ['linear', 'rbf'],

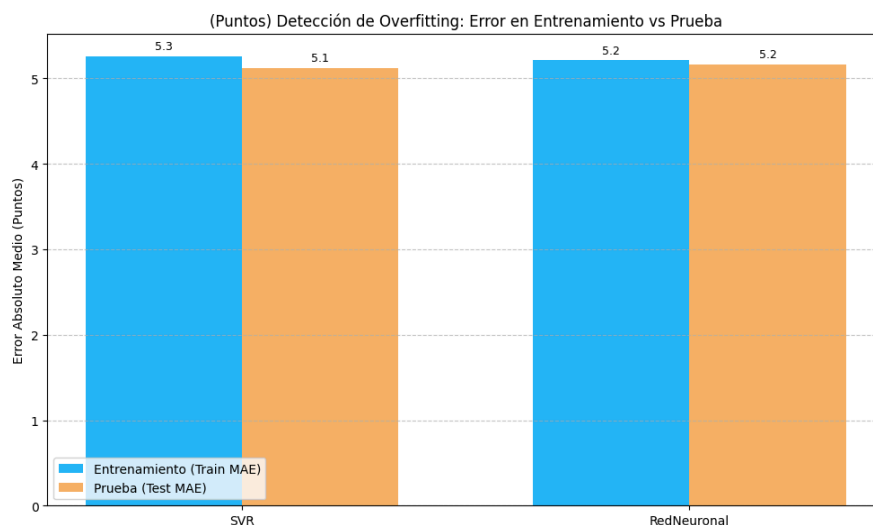
        # Ampliamos hacia valores más pequeños ya que 0.1 fue el mejor
        'regressor__C': [0.001, 0.005, 0.01, 0.05, 0.1, 0.2, 0.3, 0.5],

        # Ampliamos hacia valores más grandes ya que 0.5 fue el mejor
        'regressor__epsilon': [0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 1.0, 1.2, 1.5, 2.0]
    },
    'RedNeuronal': {
        # Exploramos arquitecturas similares a (16, 8), agregando un poco de profundidad y anchura
        'regressor__hidden_layer_sizes': [(16, 8), (32, 16, 8), (8, 8, 8), (8, 16, 32)],

        'regressor__activation': ['relu', 'tanh'],
        'regressor__alpha': [0.3, 0.4, 0.5, 1.0, 2.0],
        'regressor__learning_rate_init': [0.0001, 0.001, 0.01]
    }
}
```

- Los modelos tardaron mucho en entrenarse, sklearn definitivamente se queda muy corto en temas de redes neuronales, sobre todo porque no podemos usar GPU.
- No observamos ningún cambio en el MAE para SVR.
- Observamos una ligera degradación en el modelo de la red neuronal.

--- Resultados Finales ---						
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)	
SVR	5.26	5.12	6.67	0.54	1457.84	
RedNeuronal	5.21	5.16	6.64	0.54	1680.20	



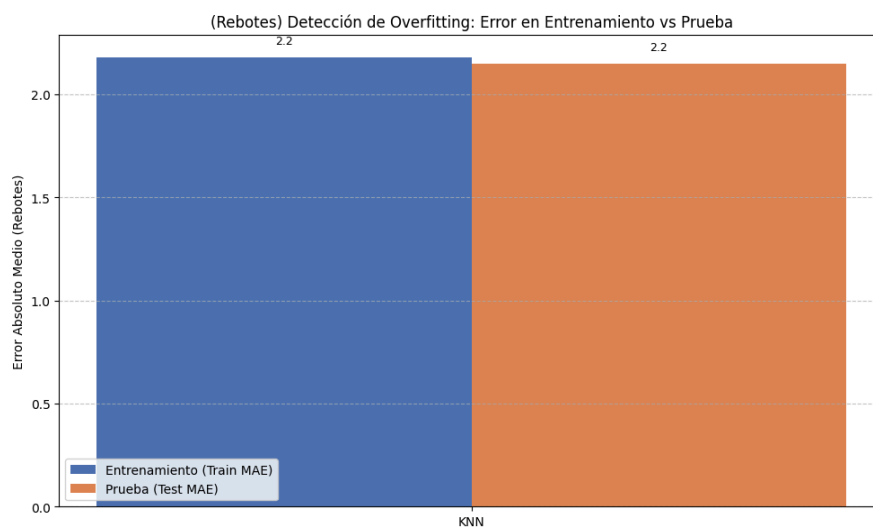
REB: KNN & Red neuronal

```
param_grids = {
    'KNN': {
        'regressor__n_neighbors': [10, 15, 20, 25, 30, 40, 50, 55],
        'regressor__weights': ['uniform', 'distance'],
        'regressor__p': [1, 2]
    },
    'RedNeuronal': {
        'regressor__hidden_layer_sizes': [(32, 16, 8), (8, 8, 8), (8, 16, 32)],
        'regressor__activation': ['relu', 'tanh'],
        'regressor__alpha': [1.0, 1.5, 2.0],
        'regressor__learning_rate_init': [0.0001, 0.001, 0.01]
    }
}
```

- La red neuronal tardó mucho en entrenarse, mismo problema que en el entrenamiento de puntos.
- KNN presenta mejoras por 0.5 decimales y su entrenamiento fue rápido.
- Observamos una ligera degradación en el modelo de la red neuronal.

KNN

--- Resultados Finales ---					
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)
KNN	2.18	2.15	2.8	0.36	26.58



Red neuronal

--- Resultados Finales ---					
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)
RedNeuronal	2.24	2.14	2.75	0.38	753.45



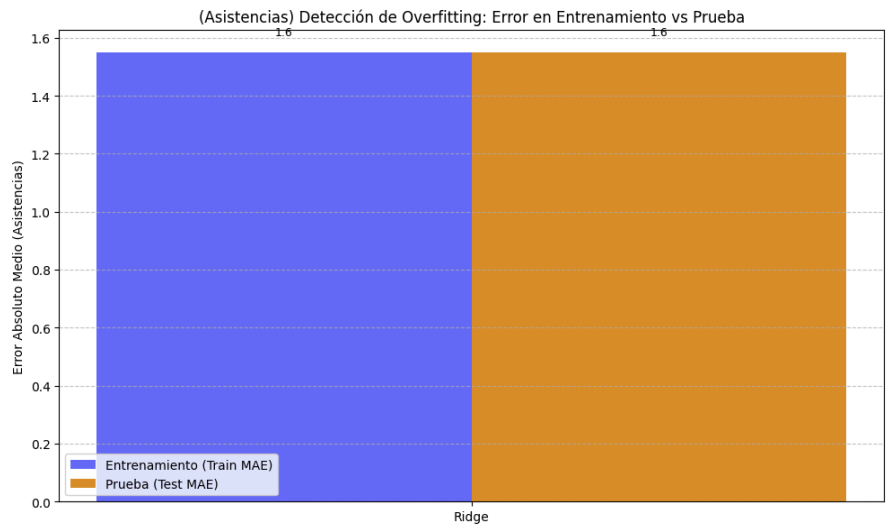
AST: Ridge & Red neuronal

```
param_grids = {
  'Ridge': {
    'regressor__alpha': [10.0, 50.0, 100.0, 200.0, 500.0, 1000.0],
    'regressor__solver': ['auto', 'svd', 'cholesky', 'lsqr', 'sag', 'saga'],
    'regressor__tol': [1e-4, 1e-3, 1e-2],
    'regressor__fit_intercept': [True, False]
  },
  'RedNeuronal': {
    'regressor__hidden_layer_sizes': [(32, 16, 8), (8, 8, 8), (8, 16, 32)],
    'regressor__activation': ['relu', 'tanh'],
    'regressor__alpha': [1.0, 1.5, 2.0],
    'regressor__learning_rate_init': [0.0001, 0.001, 0.01]
  }
}
```

- La red neuronal tardó mucho en entrenarse, mismo problema que en el entrenamiento de puntos.
- No observamos cambios en Ridge.
- Observamos una ligera mejora en el modelo de la red neuronal, es el primero en los 3 casos donde se presenta esto.

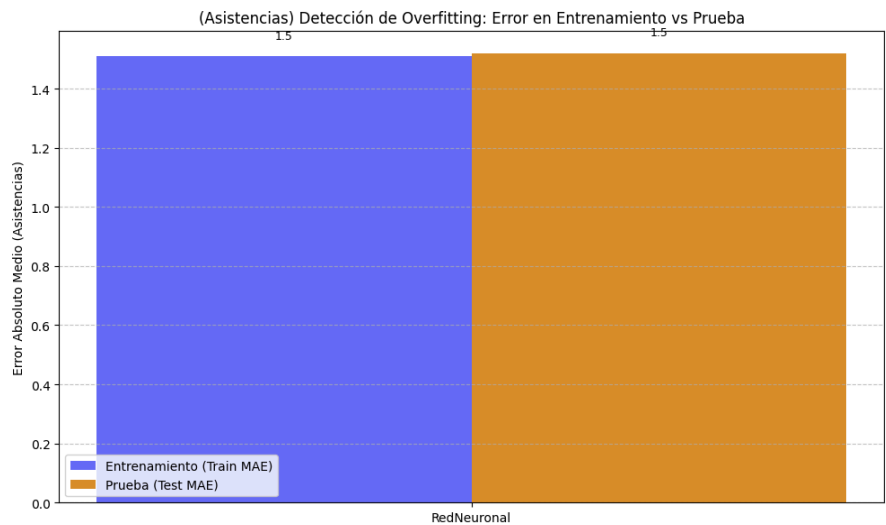
Ridge

--- Resultados Finales ---					
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)
Ridge	1.55	1.55	2.07	0.46	22.63



Red neuronal

--- Resultados Finales ---					
Modelo	Train MAE	Test MAE	Test RMSE	Test R2	Tiempo entrenamiento (s)
RedNeuronal	1.51	1.52	2.05	0.46	653.75



Elegir el modelo individual final.

A continuación se presenta una tabla comparativa con los resultados de las métricas obtenidas de los 2 mejores modelos:

PUNTOS				
Modelo	Train MAE	Test MAE	Test RMSE	Test R2
SVR	5.26	5.12	6.67	0.54
Red neuronal	5.21	5.16	6.64	0.54

REBOTES				
Modelo	Train MAE	Test MAE	Test RMSE	Test R2
KNN	2.18	2.15	2.8	0.36
Red neuronal	2.24	2.14	2.75	0.38

ASISTENCIAS				
Modelo	Train MAE	Test MAE	Test RMSE	Test R2
Ridge	1.55	1.55	2.07	0.46
Red neuronal	1.51	1.52	2.05	0.46

Definitivamente sí vimos mejoras en utilizar los datos de todos los jugadores para entrenar nuestro modelo.

Podemos ver que en los dos primeros casos parecería que una red neuronal no sería el camino óptimo a seguir, sin embargo, considerando que un SVR y una KNN tienden a utilizarse con problemas de clasificación, valdría la pena seguir explorando una red neuronal ajustando el tamaño de la misma, y los hiper parámetros con un enfoque

estocástico, incluso utilizando herramientas como optuna la cual nos puede ayudar a ir ajustando los hiper parámetros conforme se hace el entrenamiento.

Además, utilizando otra librería diferente a sklearn, podríamos aprovechar el uso de tarjetas gráficas para acelerar el entrenamiento.

Otro punto a favor es que los otros modelos (SVR, Ridge y un Lasso), dieron mejores resultados con los valores intermedios de la grilla, lo que nos indica que no valdría la pena explorar valores más grandes, si no que tendría que hacer un ajuste decimal de los hiper parámetros.

Por último, quitando la restricción de modelos de modelos ensambles, podemos explorar los árboles de decisión, ya que en los REB y AST presentaron resultados interesantes.

Pero en conclusión, creemos que una red neuronal nos puede seguir dando juego para seguir modificando y explorando hiper parámetros.

Referencias

Leidwein, F. (2025). *How to Build an ELO Rating System for the Premier League in Python*. Recuperado de:

<https://statsultra.com/how-to-build-an-elo-rating-system-for-the-premier-league-in-python/>

Amazon Web Services. (2024). Amazon Comprehend Developer Guide. <https://aws.amazon.com/comprehend/>

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Galván Bellman, J. C., Méndez González, L. C., Pérez Olguín, I. J. C., & Quezada Carreón, A. E. (2024). Algoritmo de Machine Learning para la Predicción de un Partido de Fútbol. *Revista Academia Journals*, 16(4).

GDELT Project. (2024). Global Database of Events, Language, and Tone. <https://www.gdeltproject.org/>

Google Cloud. (2024). Natural Language API Documentation. <https://cloud.google.com/natural-language>

Maher, M. J. (1982). Modelling association football scores. *Statistica Neerlandica*, 36(3), 109-118.

Microsoft Azure. (2024). Text Analytics Documentation. <https://azure.microsoft.com/en-us/products/ai-services/text-analytics/>

NewsAPI. (2024). News API Documentation. <https://newsapi.org/>

Visengeriyeva, L., Kammer, A., Bär, I., Kniesz, A., y Plöd, M. (2023). *CRISP-ML(Q). The ML Lifecycle Process*. MLOps. INNOQ. <https://ml-ops.org/content/crisp-ml>